Lab2:测试与仿真 实验报告

PB24111621 肖瀚

一、实验目的与内容

1. 进一步了解 verilog 的语法知识，重点学习如何避免出现锁存器；
2. 学习编写测试文件，学习 Testbench 的基本结构，以及时序控制、initial 和 always 、循环语句、常用系统函数等在 Testbench 中的应用；
3. 学习使用 Vivado 进行 RTL 分析与仿真，从而检验代码的正确性。

二、代码编写

【必做内容】

题目1：if 语句与锁存器

为消除锁存器，只需要将变量 shut_off_compute 和 keep_driving 初始化为0即可，代码如下：

```
module top_module (
    input                       cpu_overheated,
    output  reg                 shut_off_computer,
    input                       arrived,
    input                       gas_tank_empty,
    output  reg                 keep_driving
);
    always @(*) begin
        shut_off_computer = 1'b0;
        if (cpu_overheated)
            shut_off_computer = 1'b1;
    end

    always @(*) begin
        keep_driving = 1'b0;
        if (~arrived)
            keep_driving = ~gas_tank_empty;
    end
endmodule
```

题目2：case 语句与锁存器

为了避免产生锁存器，首先将 up, down, left, right四个变量初始化为0，然后根据扫描码的结果，利用 case 语句将对应的键位设为1，即可实现扫描码与键位的映射。实验代码如下：

```
module top_module (
    input       [15:0]                  scancode,
    output reg                          left,
    output reg                          down,
```

```
    output reg                          right,
    output reg                          up
);
    always @(*) begin
        up = 1'b0; down = 1'b0; left = 1'b0; right = 1'b0;
        case (scancode)
            16'he06b:   left = 1'b1;
            16'he072:   down = 1'b1;
            16'he074:   right = 1'b1;
            16'he075:   up = 1'b1;
            default:;
        endcase
    end
endmodule
```

题目3：计数器 Pro

- 对于问题1，我们使用 parameter 关键字指定参数 MIN_VALUE，作为计数器的最小值。
- 对于问题2，添加信号 enable 来控制计数器的工作状态，由于rst的优先级高于enable，故rst应作为首个被判断的信号。在rst有效时，计数器直接置零；在`rst == 0` 的情况下，若`enable == 1`，则增添 if-else 语句实现计数器在 MIN_VALUE 至 MAX_VALUE 之间正常工作；若`enable == 0`，则 counter 置零。另外，always语句在时钟上升沿触发，从 enable 变为低电平开始到 counter 复位并保持在 0 的间隔不超过一个时钟周期，满足题目的三个时钟周期的要求。
- 解答代码如下：

```
module Counter #(
    parameter   MAX_VALUE = 8'd100,
    parameter   MIN_VALUE = 8'd0
)(
    input                   clk,
    input                   rst,
    input                   enable,
    output                  out
);

reg [7:0] counter;
always @(posedge clk) begin
    if (rst)
        counter <= 0;
    else if (enable) begin
        if (counter < MIN_VALUE)
            counter <= MIN_VALUE;
        else if (counter >= MAX_VALUE)
            counter <= MIN_VALUE;
        else
            counter <= counter + 8'b1;
    end
    else
        counter <= 0;
end
```

```
assign out = (counter == MAX_VALUE);
endmodule
```

题目5：1的个数 pro

采用 for 循环，从第 b 位开始直到第 c 位，将 a 的每一位相加，所得结果就是数据 a 中从第 b 位到第 c 位（包含两端）这个区间内1的个数。求解代码如下：

```
module CountOnes(
    input       [31:0]         a,
    input       [4:0]          b,
    input       [4:0]          c,
    output reg  [5:0]         out
);
    integer i;
    always @(*) begin
        out = 'b0;
        for (i = b; i <= c; i = i + 1) begin
            out = out + a[i];
        end
    end
endmodule
```

【选择性必做内容】

题目2：计数器 天上天下天地无双版

使用 case 语句，在计数器达到某个值后，将它的值更改为对应格雷码的下一位的值，在达到 4 位格雷码的最大值 1000 后，将计数器置零，输出 out 为 1，如此便实现了格雷码计数器。代码如下：

```
module GrayCounter(
input                               clk,
input                               rst,
output                              out
);

reg [3:0] gray_count;

always @(posedge clk) begin
    if (rst)
        gray_count <= 4'b0000;
    else begin
        case (gray_count)
            4'b0000:    gray_count <= 4'b0001;
            4'b0001:    gray_count <= 4'b0011;
            4'b0011:    gray_count <= 4'b0010;
            4'b0010:    gray_count <= 4'b0110;
            4'b0110:    gray_count <= 4'b0111;
            4'b0111:    gray_count <= 4'b0101;
```

```
            4'b0101:    gray_count <= 4'b0100;
            4'b0100:    gray_count <= 4'b1100;
            4'b1100:    gray_count <= 4'b1101;
            4'b1101:    gray_count <= 4'b1111;
            4'b1111:    gray_count <= 4'b1110;
            4'b1110:    gray_count <= 4'b1010;
            4'b1010:    gray_count <= 4'b1011;
            4'b1011:    gray_count <= 4'b1001;
            4'b1001:    gray_count <= 4'b1000;
            4'b1000:    gray_count <= 4'b0000;
        endcase
    end
end

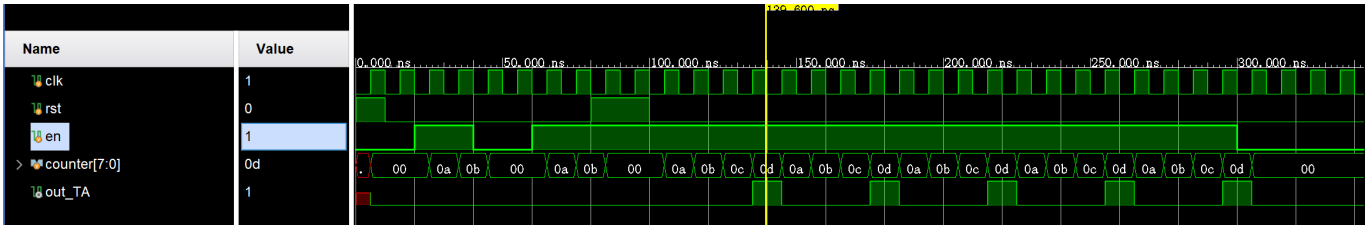
assign out = (gray_count == 4'b1000);

endmodule
```

三、仿真结果与分析

1. **必做题目3的仿真**

这里借用的是助教的仿真文件，仿真结果如下图：



可以看到计数器在 enable 信号有效时，会在 0a ~ 0d(10 ~ 13) 之间循环计数，每到一次 0d ，out 就会输出为 1。

2. **必做题目5的输出结果**

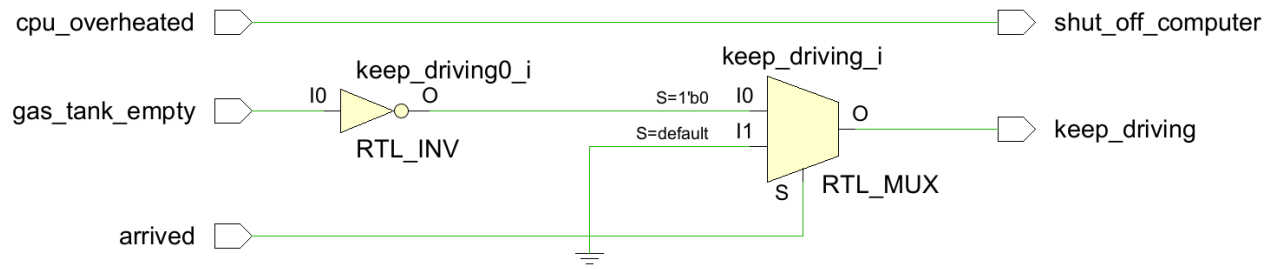
这里同样用到了助教的测试文件，测试结果如下：

```
TEST SUMMARY
=========================================
Total tests:  40
Passed:       40
Failed:       0
SUCCESS: All tests passed!
$finish called at time : 800 ns : File "D:/vivadoproj/Lab2/Lab2.srcs/sim_1/new/CountOnes_tb.v" Line 134
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_CountOnes_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```

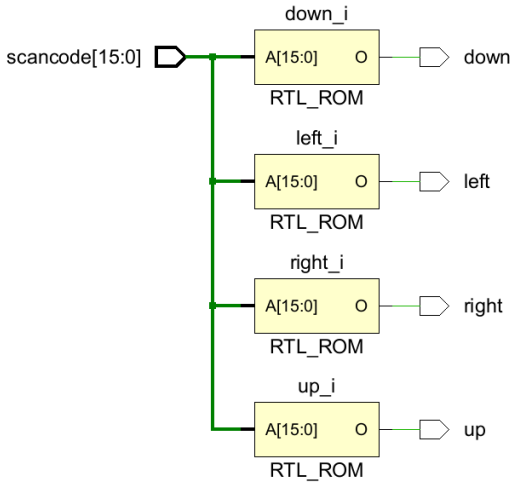
显示 40 个测试数据全部通过，检验了代码的正确性。

四、电路设计与分析

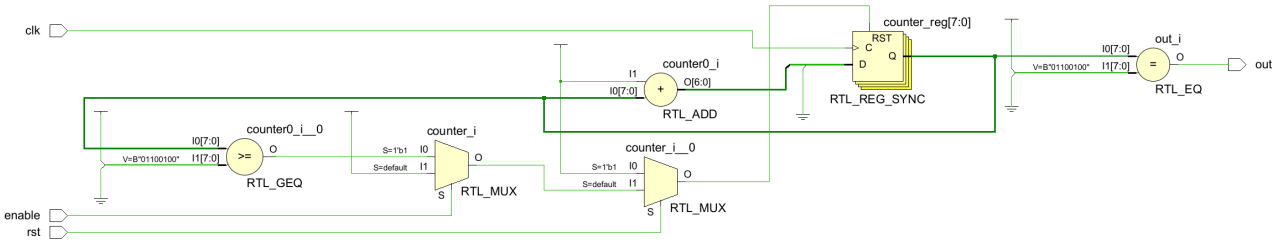
1. 必做题目1的 RTL 电路图如下，电路中含有一个反相器和一个选择器，没有产生锁存器。



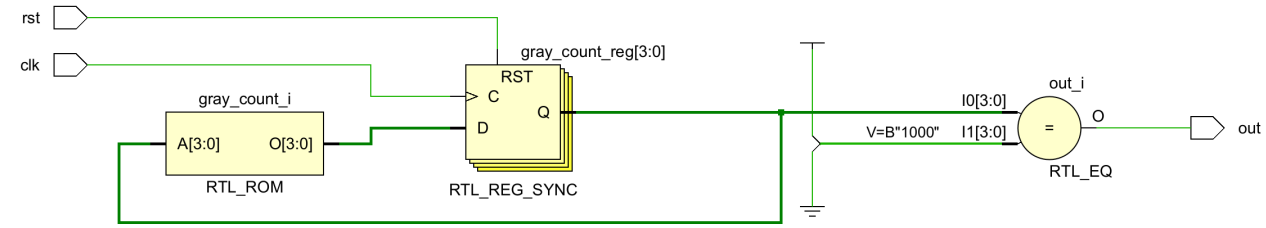
2. 必做题目2的 RTL 电路图如下，可以看到有四个只读存储器连接对应的键位输出，同样没有锁存器。



3. 必做题目3的 RTL 电路图如下，计数器在每个时钟周期递增，其输出值用于选择或计算处理对象,处理结果通过加法器等运算单元进行累积，并暂存在同步寄存器(RTL_REG_SYNC)中；多路复用器(RTL_MUX)则用于控制数据路径的选择；当比较器判断计数器达到预设条件时，会生成相应的控制信号。



4. 选择性必做题目2的 RTL 电路图如下，该电路包括三个部分：RTL_ROM（只读存储器）、RTL_REG_SYNC（同步寄存器）、RTL_EQ（等于比较器）。



五、总结

通过本次实验，我对 Verilog 语法有了进一步的认识，知道在编写Verilog代码的时候，要尽可能避免产生锁存器；也对测试文件 Testbench 有了初步了解，学会用 Testbench 文件对编写的 Verilog 代码进行仿真，以及利用 Vivado 生成对应的 RTL 电路图，来检验代码的正确性。